BOB-075 followup â€” concurrency guard anti-bluff evidence

Revision: 1 **Last modified:** 2026-08-18T15:00:00Z

Followup from BOB-075 SDD task review (subagent a584f8cc): the reviewer inadvertently corrupted `docs/codegraph/Status.md` by running two concurrent `status_docs_freshness_challenge.sh` invocations (the backdate+trap+restore was not thread-safe). This directory carries the Â§11.4.115 polarity proof for the fix: real corruption BEFORE the fix, real lock rejection AFTER the fix, and a Â§1.1 mutation proving the lock is load-bearing.

All reproductions below run against a **scratch harness** (a throwaway project-root directory containing only the challenge script, a stub `docs_chain` engine binary that `sleep`s to simulate the real engineâ€™s wall-clock, and synthetic `docs/codegraph/Status.md` / `docs/features/Status.md` fixtures) â€” never against this repositoryâ€™s real, committed `docs/codegraph/Status.md`. The scratch harness exercises the exact same code path (same script, same directory layout relative to `PROJECT_ROOT`, same backdate+trap+restore logic) without any risk of leaving the real committed doc corrupted if a repro run goes wrong.

Methodology

1. Build a scratch project root: `<scratch>/challenges/scripts/status_docs_freshness_challenge.sh` (a copy of the script under test), `<scratch>/constitution/submodules/docs_chain/docs_chain` (a stub engine â€” `sleep "${STUB_DOCS_CHAIN_SLEEP:-N}"; echo ...; exit 0` â€” standing in for the real engineâ€™s genuine 70-90s `verify --all` wall-clock, which is exactly what gives two manually-started shells in real usage enough time to overlap), and fresh `docs/codegraph/Status.md` / `docs/features/Status.md` fixtures with an in-SLA **Last modified:** header.
2. Launch `RED_MODE=1 bash challenges/scripts/status_docs_freshness_challenge.sh` twice in the background, staggered by a small, swept delay (0s-2.5s), from that scratch root.
3. Wait for both to exit; diff the final `docs/codegraph/Status.md` against the pre-run original. A byte-identical result is `CLEAN`; any difference (in every observed case, the **Last modified:** header stuck at the backdated â€œ100 days agoâ€ value) is `CORRUPTED`.

No synchronization hooks, sleeps, or barriers were injected into the script under test anywhere in this evidence â€” every reproduction is genuine, unforced concurrent execution (â€œrun 2 shells simultaneouslyâ€, per the task brief), with only the launch stagger swept to find a hit.

Results summary

Scenario	Script variant	Lock available	Trials	Corrupted
`before_fix/`	HEAD (pre-fix)	n/a â€” no lock exists	15 across two sweeps (10-stagger exploratory + 5-trial repeat at the hit stagger)	8/15 (see `before_fix/reliability_sweep_sum`
`after_fix/`	fixed	flock	6 (staggers 0-1.0s)	0/6
`mkdir_fallback/`	fixed	mkdir (flock hidden via `PATH`)	3 attempts (2 hit unrelated fakebin `PATH`-harness gaps â€” see `mkdir_fallback/notes.txt` â€” but still correctly rejected the 2nd instance in both; 1 fully clean end-to-end pass saved here)	0
`signal_handling/`	fixed	flock	1 (SIGTERM sent mid-backdate)	0 (file restored, lock rele that order)
`mutation_test/`	fixed **with `acquire_lock` call commented out**	none (mutated)	20 (dense stagger sweep)	2/20 â€” see `mutation_t notes.txt`

Polarity confirmed (Â§11.4.115): unguarded concurrent execution corrupts the file (reproducible, `before_fix/`); guarded concurrent execution never does across 26 combined trials (`after_fix/` + `mkdir_fallback/`); stripping *only* the lock invocation from the fixed script (the Â§1.1 mutation) restores the ability to corrupt (`mutation_test/`) â€” proving the lock, not some incidental side effect, is what prevents the corruption.

Honesty note on hit rate (Â§11.4.6): this is a genuine OS-scheduler race, not a deterministic bug â€” the exact hit rate varies with host load and with incidental script-length/parse-timing changes (the fixed script is longer, which measurably shifts the race window; see `mutation_test/notes.txt` for detail). What is invariant across every sweep run for this evidence package is the *polarity*: 0 corruptions observed with the lock engaged, and repeated corruptions observed with it absent (either pre-fix or via the mutation).

Directory contents

- `before_fix/` â€” unmodified (git HEAD, pre-fix) script. `instance_A.log` / `instance_B.log` are the two concurrent runsâ€™ full transcripts for one representative corrupted trial (stagger=0.3s). `original_codegraph_status.md` vs `final_codegraph_status_CORRUPTED.md` / `diff_original_vs_final.diff` show the corruption directly. `reliability_sweep_summary.txt` records 5 repeated trials at the same stagger (3/5 corrupted) plus an earlier 10-stagger exploratory sweep (3/10 corrupted at staggers 0s/0.05s/0.3s).
- `after_fix/` â€” fixed script, flock available. `instance_A_winner.log` completes the full baseline+RED+GREEN cycle and exits 0. `instance_B_rejected.log` fails fast with the actionable lock-held message and exits 1 *before touching the file*. `stagger_sweep_summary.txt` covers 6 staggers, all clean.
- `mkdir_fallback/` â€” same test with `PATH` restricted to a directory with no `flock` binary, forcing the portable mkdir-based lock. Same outcome (one winner, one fast-rejected loser, clean final state, no stray lock artifacts).
- `signal_handling/` â€” a single long-running instance (8s stub sleep to widen the window), polled until its backdate was confirmed live on disk, then sent `SIGTERM`. `victim.log` + `notes.txt` show the file was restored to the true original *before* the process exited, and no lock artifact was left behind â€” the required â€œrestore first, then release the lockâ€ ordering.
- `mutation_test/` â€” the fixed script with the single `acquire_lock` invocation commented out (`mutation.diff`). A 20-trial dense stagger sweep reproduces corruption twice (`instance_A.log` / `instance_B.log` / `diff_original_vs_final.diff` are the idx=5 hit); `notes.txt` has the full sweep breakdown and the honesty note on hit-rate variance.

Fix reference

The concurrency guard lives in `challenges/scripts/status_docs_freshness_challenge.sh`: `acquire_lock()` / `release_lock()` (flock on the scriptâ€™s own file, mkdir fallback with Â§11.4.180 stale-holder-PID reaping for portability), invoked once near the top of the script before any file mutation, and a unified `cleanup()` trap (registered `EXIT INT TERM`) that restores the real file first (if a backdate is in flight) and only then releases the lock â€” see the scriptâ€™s own top-of-file doc block for the full design rationale.